

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Design and Analysis of Algorithms - Assignment 1 (CO1)

Item	Details
Student name	Natesa Krishnan V
Registration number	192511311
Group	A (third algorithm: Hash Table Search)
Personal dataset seed	1311
Search keys	FOUND key = 192511311 ; NOT-FOUND key = 192511312
Algorithms studied	1. Linear Search (iterative) 2. Recursive Binary Search 3. Hash Table Search
Dataset sizes	1,000 / 10,000 / 100,000 / 1,000,000 registration numbers
Implementation	Python 3.12.3 (Linux x86_64)
Code repository folder	DAA-A1-192511311-Natesa-Krishnan-V

All execution times, comparison counts and graphs in this report were produced by running the programs in the accompanying project (src/) on the generated datasets (data/); the raw numbers are in results/results.csv.

Task 1 - Algorithm Design

The problem: given a registration number (key) and a collection of n student registration numbers, decide whether the key is present and, if so, return its position. n grows from 1,000 to 1,000,000+. Any preprocessing (sorting, building an index) must be charged to the algorithm.

Table 1.1 - The three selected strategies

	Algorithm 1	Algorithm 2	Algorithm 3
Name	Linear Search	Recursive Binary Search	Hash Table Search (Group A)
Design strategy	Brute force (exhaustive scan)	Decrease-and-conquer (decrease by a factor of 2)	Space-time trade-off (input enhancement by hashing)
Style	Iterative	Recursive	Direct addressing + chain scan
Preprocessing	None	Sort the records: $\Theta(n \log n)$	Build the table: n insertions, $\Theta(n)$
Requires sorted data?	No	Yes	No
Extra memory	$O(1)$	$O(\log n)$ recursion stack (array sorted in a copy)	$\Theta(n)$ for table and chains

The three approaches are different strategies, not variants of one algorithm: brute force inspects records one by one, decrease-and-conquer exploits ordering to discard half of the remaining records per step, and hashing exploits extra memory to jump straight to the place where the key must be.

1.1 Algorithm 1 - Linear Search (iterative, brute force)

Basic idea: examine the records from the first to the last and compare each with the key; stop when a match is found, or report failure when all n records have been examined. No assumptions are made about the order of the data.

```
ALGORITHM LinearSearch(A[0..n-1], key)
// Input : array A of n registration numbers (unsorted), search key
// Output: index of key in A, or -1 if absent
for i <- 0 to n-1 do
    if A[i] = key then      // basic operation: key comparison
        return i
return -1
```

Justification: it is the natural baseline - it needs no preprocessing, handles unsorted data and is the cheapest choice when only a handful of searches are made. Investigating it shows exactly how badly a $\Theta(n)$ scan scales to 1,000,000+ records.

1.2 Algorithm 2 - Recursive Binary Search (decrease-and-conquer)

Basic idea: on a SORTED array, compare the key with the middle element. If equal, done; if the key is smaller, the answer can only be in the left half; if larger, only in the right half. Each call therefore discards about half of the remaining records and makes exactly one recursive call. Preprocessing: the records must first be sorted ($\Theta(n \log n)$); this cost is measured separately and added to the total.

```
ALGORITHM BinarySearch(A[lo..hi], key)      // A is sorted ascending
if lo > hi then return -1                    // base case: empty range
mid <- floor((lo + hi) / 2)
if A[mid] = key then return mid              // basic operation: key comparison
if A[mid] < key then
    return BinarySearch(A, mid+1, hi, key)  // right half
else
    return BinarySearch(A, lo, mid-1, key)  // left half
```

```
PREPROCESSING: A <- sort(records)          //  $\Theta(n \log n)$ 
SEARCH:        BinarySearch(A, 0, n-1, key)
```

Flowchart (text form)

```
START --> [lo > hi ?] --yes--> RETURN -1
          | no
```

```

v
[ mid = (lo+hi)/2 ]
v
[ A[mid] = key ? ] --yes--> RETURN mid
| no
v
[ A[mid] < key ? ] --yes--> recurse on (mid+1 .. hi) --+
| no
v
recurse on (lo .. mid-1) -----+--> RETURN result

```

Justification: it is the classic logarithmic search; it is the only one of the three that is recursive and it produces the recurrence analysed in Tasks 6 and 7. It also exposes the central design question of this assignment - whether the one-off sorting cost is repaid by the cheaper searches.

1.3 Algorithm 3 - Hash Table Search (Group A, space-time trade-off)

Basic idea: store every registration number in a hash table with separate chaining. The hash function $h(\text{key}) = \text{key} \bmod m$ (m prime, $m \geq n/0.75$, so the load factor $\alpha = n/m$ is about 0.75) maps a key directly to one bucket; only the short chain in that bucket is scanned. Preprocessing: insert all n records (each insertion is an $O(1)$ append to a chain) - $\Theta(n)$ time and $\Theta(n)$ extra memory.

```

PREPROCESSING BuildTable(A[0..n-1])
  m <- smallest prime >= n / 0.75
  T[0..m-1] <- empty chains
  for i <- 0 to n-1 do
    T[ A[i] mod m ].append( (A[i], i) )    // O(1)

ALGORITHM HashSearch(T, key)
  b <- T[ key mod m ]                      // 1 hash computation
  for each (k, idx) in b do
    if k = key then return idx              // basic operation: key comparison
  return -1

```

Justification: the assignment's Group-A requirement, and the strategy that trades memory for speed. It is expected to beat both other approaches per search, but its build cost and memory use must be weighed against the number of searches that will actually be made.

Task 2 - Identify the Basic Operation

Table 2.1 - Basic operation of each algorithm

Algorithm	Basic operation	Where it occurs	Why dominant	How its count changes with n
Linear Search	Key comparison $A[i] = \text{key}$	Inside the for loop, once per iteration	Every iteration does one comparison plus loop bookkeeping that is proportional to it; no other step is repeated as often.	Grows linearly: 1 (best) to n (worst); about $n/2$ on average for a successful search.
Recursive Binary Search	Key comparison with $A[\text{mid}]$	Once in every recursive call that has a non-empty range	Each call does one comparison and $O(1)$ other work (computing mid, one branch); the number of calls equals the number of comparisons.	Grows logarithmically: at most $\text{floor}(\log_2 n) + 1$; doubling n adds one comparison.
Hash Table Search	Key comparison inside the bucket chain (after one hash computation)	In the chain-scan loop of HashSearch	Hashing itself is a fixed $O(1)$ step; the only part whose cost depends on the data is the chain scan.	Stays almost constant: expected about α (unsuccessful) or $1 + \alpha/2$ (successful) comparisons, independent of n when α is bounded.

Why comparisons are a valid yardstick: in all three algorithms the remaining work per step (loop index update, computing mid, computing $\text{key} \bmod m$, following a list pointer) is a constant multiple of the comparison count, so running time is proportional to the comparison count and the count captures the growth with n . For the two algorithms with preprocessing, the basic operation of the preprocessing step is

also counted separately: element comparisons during sorting ($\Theta(n \log n)$) and element insertions during table construction ($\Theta(n)$). In the experiments the count is recorded directly by the programs (comparisons / operations columns in the CSV files).

Task 3 - Mathematical Time-Complexity Function

Input size: n = number of student records. Let q be the number of searches performed after preprocessing. Let c denote the (constant) cost of one basic operation including its loop overhead.

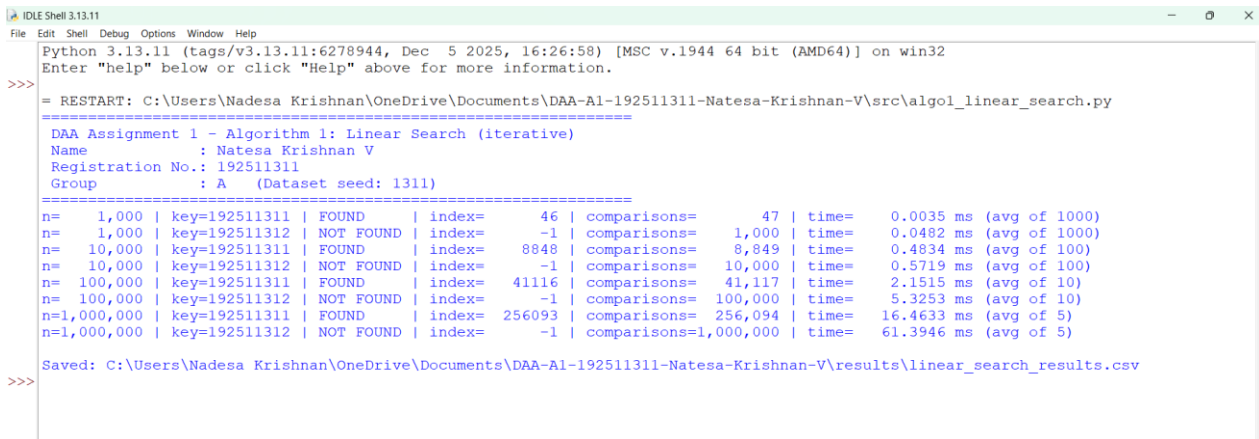
3.1 Linear Search

The loop body executes once per examined record. If the key is at (0-based) position j , exactly $j + 1$ comparisons are made; if the key is absent, all n records are examined. Counting the executions of the basic operation with a summation:

$$C_{\text{unsuccessful}}(n) = \sum_{i=0}^{n-1} 1 = n \quad C_{\text{successful}}(j) = \sum_{i=0}^j 1 = j + 1$$

Hence $T(n) = c \cdot C(n) + d$, where d is the constant cost of initialisation and return. In the worst case $T(n) = c \cdot n + d$, which is a linear function of n . Preprocessing cost: 0. Total cost of q searches:

$$T_{\text{total}}(n, q) = q * (c \cdot n + d) = \Theta(q \cdot n)$$



```

Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\src\algo1_linear_search.py

=====
DAA Assignment 1 - Algorithm 1: Linear Search (iterative)
Name      : Natesa Krishnan V
Registration No.: 192511311
Group     : A      (Dataset seed: 1311)
=====
n= 1,000 | key=192511311 | FOUND | index= 46 | comparisons= 47 | time= 0.0035 ms (avg of 1000)
n= 1,000 | key=192511312 | NOT FOUND | index= -1 | comparisons= 1,000 | time= 0.0482 ms (avg of 1000)
n= 10,000 | key=192511311 | FOUND | index= 8848 | comparisons= 8,849 | time= 0.4834 ms (avg of 100)
n= 10,000 | key=192511312 | NOT FOUND | index= -1 | comparisons= 10,000 | time= 0.5719 ms (avg of 100)
n= 100,000 | key=192511311 | FOUND | index= 41116 | comparisons= 41,117 | time= 2.1515 ms (avg of 10)
n= 100,000 | key=192511312 | NOT FOUND | index= -1 | comparisons= 100,000 | time= 5.3253 ms (avg of 10)
n=1,000,000 | key=192511311 | FOUND | index= 256093 | comparisons= 256,094 | time= 16.4633 ms (avg of 5)
n=1,000,000 | key=192511312 | NOT FOUND | index= -1 | comparisons=1,000,000 | time= 61.3946 ms (avg of 5)

Saved: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\results\linear_search_results.csv
>>>

```

3.2 Recursive Binary Search

Searching: each call performs one comparison and then recurses on a sub-array of at most $\text{floor}(n/2)$ elements (derived in Task 6). Starting with n elements, after i calls at most $\text{floor}(n / 2^i)$ elements remain; the search must stop when no elements remain, i.e. when $\text{floor}(n / 2^i) = 0$, which first happens for $i = \text{floor}(\log_2 n) + 1$. Therefore the number of comparisons in the worst case is

$$C_{\text{worst}}(n) = \text{floor}(\log_2 n) + 1 \Rightarrow T_{\text{search}}(n) = c \cdot (\text{floor}(\log_2 n) + 1) + d = \Theta(\log n)$$

Preprocessing: the array must be sorted. Any comparison-based sort needs at least $\log_2(n!)$ comparisons, and $\log_2(n!) = \Theta(n \log n)$ (Stirling: $\log_2(n!) = n \log_2 n - n \log_2 e + O(\log n)$). The sort used (Python's Timsort, a merge-sort variant) is $O(n \log n)$ in the worst case, so

$$T_{\text{pre}}(n) = a \cdot n \log_2 n + b \cdot n = \Theta(n \log n)$$

Overall cost of sorting once and then performing q searches:

$$T_{\text{total}}(n, q) = a \cdot n \log_2 n + q \cdot c \cdot (\log_2 n + 1) = \Theta((n + q) \log n)$$

```

IDLE Shell 3.13.11
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\src\algo2_recursive_binary_search.py
=====
DAA Assignment 1 - Algorithm 2: Recursive Binary Search
Name      : Natesa Krishnan V
Registration No.: 192511311
Group     : A (Dataset seed: 1311)
=====
n= 1,000 | key=192511311 | FOUND | index(sorted)= 756 | comparisons= 7 | sort= 0.0571 ms | search= 0.001125 ms | tot
al= 0.0583 ms
n= 1,000 | key=192511312 | NOT FOUND | index(sorted)= -1 | comparisons= 10 | sort= 0.0571 ms | search= 0.001648 ms | tot
al= 0.0588 ms
n= 10,000 | key=192511311 | FOUND | index(sorted)= 7595 | comparisons= 14 | sort= 1.1674 ms | search= 0.002282 ms | tot
al= 1.1697 ms
n= 10,000 | key=192511312 | NOT FOUND | index(sorted)= -1 | comparisons= 14 | sort= 1.1674 ms | search= 0.002879 ms | tot
al= 1.1703 ms
n= 100,000 | key=192511311 | FOUND | index(sorted)= 75598 | comparisons= 17 | sort= 15.5369 ms | search= 0.003038 ms | tot
al= 15.5399 ms
n= 100,000 | key=192511312 | NOT FOUND | index(sorted)= -1 | comparisons= 17 | sort= 15.5369 ms | search= 0.004256 ms | tot
al= 15.5412 ms
n=1,000,000 | key=192511311 | FOUND | index(sorted)= 755937 | comparisons= 18 | sort= 264.1240 ms | search= 0.004800 ms | tot
al= 264.1288 ms
n=1,000,000 | key=192511312 | NOT FOUND | index(sorted)= -1 | comparisons= 20 | sort= 264.1240 ms | search= 0.003241 ms | tot
al= 264.1272 ms
Saved: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\results\binary_search_results.csv
>>>

```

3.3 Hash Table Search

Building: n insertions, each costing a constant (compute key mod m , append to a chain), plus allocating $m = \Theta(n)$ buckets:

$$T_{\text{pre}}(n) = n * c_{\text{ins}} + m * c_{\text{alloc}} = \Theta(n)$$

Searching: one hash computation (constant) plus a scan of one chain. Under the assumption of simple uniform hashing (every key is equally likely to fall in any of the m buckets - reasonable here because the registration numbers are pseudo-random), the expected chain length is $\alpha = n/m$. The expected number of comparisons for an unsuccessful search is therefore α . For a successful search for the i -th inserted key, the keys that precede it in its chain are those inserted earlier into the same bucket, $(i-1)/m$ on average, so it needs $1 + (i-1)/m$ comparisons; averaging over all keys:

$$C_{\text{successful}} = (1/n) * \sum_{i=1}^n (1 + (i-1)/m) = 1 + (n-1)/(2m) = 1 + \alpha/2 - \alpha/(2n)$$

Because α is held at about 0.75 (a constant independent of n), $T_{\text{search}} = c * (1 + \alpha) + d = \Theta(1)$ expected. In the worst case all n keys fall into one bucket and the search scans a chain of length n : $\Theta(n)$. Overall cost of building once and then performing q searches:

$$T_{\text{total}}(n, q) = n * c_{\text{ins}} + q * c * (1 + \alpha) = \Theta(n + q) \text{ (expected)}$$

```

IDLE Shell 3.13.11
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\src\algo3_hash_table.py
=====
DAA Assignment 1 - Algorithm 3: Hash Table Search (Group A)
Name      : Natesa Krishnan V
Registration No.: 192511311
Group     : A (Dataset seed: 1311)
=====
n= 1,000 | key=192511311 | FOUND | index/key= 46 | ops= 2 | build= 0.2126 ms | search= 0.000179 ms | total= 0.21
28 ms
n= 1,000 | key=192511312 | NOT FOUND | index/key=192511312 | ops= 2 | build= 0.2126 ms | search= 0.000181 ms | total= 0.21
28 ms
n= 10,000 | key=192511311 | FOUND | index/key= 8848 | ops= 2 | build= 2.3833 ms | search= 0.000184 ms | total= 2.38
34 ms
n= 10,000 | key=192511312 | NOT FOUND | index/key=192511312 | ops= 2 | build= 2.3833 ms | search= 0.000396 ms | total= 2.38
37 ms
n= 100,000 | key=192511311 | FOUND | index/key= 41116 | ops= 2 | build= 28.8423 ms | search= 0.000407 ms | total= 28.84
27 ms
n= 100,000 | key=192511312 | NOT FOUND | index/key=192511312 | ops= 1 | build= 28.8423 ms | search= 0.000292 ms | total= 28.84
26 ms
n=1,000,000 | key=192511311 | FOUND | index/key= 256093 | ops= 2 | build= 529.1889 ms | search= 0.000567 ms | total= 529.18
95 ms
n=1,000,000 | key=192511312 | NOT FOUND | index/key=192511312 | ops= 2 | build= 529.1889 ms | search= 0.000354 ms | total= 529.18
93 ms
Saved: C:\Users\Nadesa Krishnan\OneDrive\Documents\DAA-A1-192511311-Natesa-Krishnan-V\results\hash_table_results.csv
>>>

```

Table 3.1 - Summary of the derived cost functions

Algorithm	Preprocessing	One search (worst)	q searches + preprocessing
Linear Search	0	$c * n + d$	$q * n = \Theta(q n)$
Recursive Binary Search	$a n \log_2 n$	$c (\text{floor}(\log_2 n) + 1)$	$n \log n + q \log n$
Hash Table Search	$c_{\text{ins}} * n$	expected $c(1 + \alpha)$; worst $c * n$	$n + q$ (expected)

Task 4 - Best, Average and Worst-Case Behaviour

4.1 Linear Search

- **Best case:** the key is stored in the very first position. The first comparison succeeds, so $C_{\text{best}} = 1 = \Theta(1)$. The scan stops at once because the loop exits on the first match.
- **Worst case:** the key is in the last position or is absent. Every record has to be examined because nothing about the data lets the algorithm rule out a record without looking at it: $C_{\text{worst}} = n$.
- **Average case:** assume a successful search where the key is equally likely to be at any of the n positions (probability $1/n$ each). Then $C_{\text{avg}} = (1/n) * \sum_{j=1}^n j = (n+1)/2$. If the key is present with probability p and absent with probability $1-p$, $C_{\text{avg}} = p(n+1)/2 + (1-p)n$, which lies between $(n+1)/2$ and n . In all cases it is linear in n .

4.2 Recursive Binary Search

- **Best case:** the key is the middle element of the sorted array - the first comparison succeeds: $C_{\text{best}} = 1$.
- **Worst case:** the key is absent, or is found only when the range has been reduced to a single element. Each failed comparison halves the range, so the search continues for $\lfloor \log_2 n \rfloor + 1$ comparisons: $C_{\text{worst}} = \lfloor \log_2 n \rfloor + 1$.
- **Average case (successful):** take $n = 2^k - 1$. The tree of possible midpoints has $2^{(i-1)}$ elements at depth i , each found with i comparisons, so $C_{\text{avg}} = (1/n) * \sum_{i=1}^k i * 2^{(i-1)} = ((k-1)*2^k + 1)/(2^k - 1)$, which is about $k - 1 = \log_2(n+1) - 1$. An unsuccessful search needs about k comparisons. Both are $\Theta(\log n)$: about half of all elements sit in the last level of the tree, so even the average search is only one comparison cheaper than the worst.
- **Note on preprocessing:** the sort is NOT input-independent in practice - Timsort needs only about n comparisons if the data is already sorted (best case $\Theta(n)$) but $\Theta(n \log n)$ for random data, which is what the generated datasets contain.

4.3 Hash Table Search

- **Best case:** the hash bucket is empty (unsuccessful search, 0 comparisons) or the key is the first element of its chain (successful, 1 comparison). Cost: one hash computation, $\Theta(1)$.
- **Average case:** under simple uniform hashing the expected chain length is $\alpha = n/m$: expected comparisons α for an unsuccessful and $1 + \alpha/2 - \alpha/(2n)$ for a successful search (derived in Task 3). With $\alpha \sim 0.75$ this is below 2 comparisons, independent of n : $\Theta(1)$.
- **Worst case:** every key hashes to the same bucket (a poor hash function or an adversarial/structured set of keys such as all multiples of m). The chain has n elements and the search degenerates to a linear scan: $\Theta(n)$. The worst case does not depend on the key searched, only on how the data was distributed over the buckets; with pseudo-random registration numbers and a prime table size it is extremely unlikely - in the experiments the longest chain was only 5 ($n=1,000$), 6 ($n=10,000$), 6 ($n=100,000$), 2 ($n=1,000,000$).

Task 5 - Asymptotic Analysis

Table 5.1 - Asymptotic summary

Quantity	Linear Search	Recursive Binary Search	Hash Table Search
Search - best case	$\Omega(1)$, $\Theta(1)$	$\Omega(1)$, $\Theta(1)$	$\Omega(1)$, $\Theta(1)$
Search - average case	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$ expected
Search - worst case	$\Theta(n)$ (so $O(n)$)	$\Theta(\log n)$ (so $O(\log n)$)	$\Theta(n)$ (so $O(n)$)
Preprocessing	none	$\Theta(n \log n)$	$\Theta(n)$
q searches incl. preprocessing	$\Theta(q n)$	$\Theta((n + q) \log n)$	$\Theta(n + q)$ expected
Extra space	$O(1)$	$O(\log n)$ stack	$\Theta(n)$

Justification of the notation. Linear search: for the worst case the operation count is exactly n , so $n \leq T(n) \leq n$ which gives $O(n)$ and $\Omega(n)$, hence $\Theta(n)$ with $c_1 = c_2 = 1$; but over ALL inputs the running time is only bounded below by a constant ($\Omega(1)$), so the algorithm as a whole is $O(n)$ and $\Omega(1)$ and has no single Θ - $\Theta(n)$ applies to its worst and average case. Binary search: $\lfloor \log_2 n \rfloor + 1$ lies between $\log_2 n$ and $2 \log_2 n$ for $n \geq 2$, so the worst case is $\Theta(\log n)$. Hash search: the expected time $1 + \alpha$ is bounded by constants ($1 \leq 1 + \alpha \leq 2$ for $\alpha \leq 1$), so it is $\Theta(1)$ in expectation, while the worst case is $\Theta(n)$ since a chain may hold all n keys.

Growth rates: $1 \ll \log n \ll n$ as $n \rightarrow \infty$, because $\lim \log n / n = 0$ and $\lim 1 / \log n = 0$. Hence $O(1)$ (hash, expected) grows slower than $O(\log n)$ (binary), which grows slower than $O(n)$ (linear).

What this means for the university system. When the database grows from 1,000 to 1,000,000 records ($\times 1000$), the number of comparisons of a linear search also grows by $\times 1000$ (worst case 1,000 $\rightarrow$ 1,000,000). For binary search it grows from about 10 to 20 comparisons - twice as many for a thousand times more data. For the hash table it stays at about 1-2 operations. For the one-off preprocessing the picture is reversed: the sort scales as $n \log n$ and the table build as n , so a bigger database means a proportionally bigger up-front cost. Asymptotically, the logarithmic and constant-time searches are the only practical options for 1,000,000+ records if many searches are performed; the choice among them depends on how many searches share one preprocessing step (Tasks 9-12).

Task 6 - Recurrence Relation (Recursive Binary Search)

The recurrence is derived from the recursive implementation given in Task 1 (function `binary_search` in `src/algo2_recursive_binary_search.py`), not quoted from a textbook.

- **Input size of the original problem:** $n = hi - lo + 1$, the number of elements in the range being searched.
- **Number of recursive calls:** exactly ONE. After comparing with $A[mid]$, the algorithm continues either in the right half or in the left half, never both (the if/else makes the two calls mutually exclusive).
- **Size of the subproblem:** $mid = lo + \lfloor (n-1)/2 \rfloor$. The left range $A[lo..mid-1]$ contains $\lfloor (n-1)/2 \rfloor$ elements; the right range $A[mid+1..hi]$ contains $n - 1 - \lfloor (n-1)/2 \rfloor = \lfloor n/2 \rfloor$ elements. In the worst case (the key is not found, the larger half is always chosen) the subproblem size is $\lfloor n/2 \rfloor$.
- **Work outside the recursive call:** computing mid , one three-way comparison of the key with $A[mid]$ and one branch: constant work, $\Theta(1)$ - counted as 1 key comparison.
- **Base case:** $lo > hi$, i.e. $n = 0$: the range is empty and the function returns -1 immediately without comparing anything: $T(0) = 0$ comparisons (constant time).

Combining these components, the number of key comparisons in the worst case satisfies

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \quad T(0) = 0$$

and for the running time, with c the constant cost of the non-recursive work and d the cost of the base case:

$$T(n) = T(\lfloor n/2 \rfloor) + c \quad \text{for } n \geq 1, \quad T(0) = d$$

This is a divide-and-conquer recurrence of the form $T(n) = a T(n/b) + f(n)$ with $a = 1$ (one subproblem), $b = 2$ (half size), $f(n) = \Theta(1)$. Note that it is the recurrence of the SEARCH only: the sorting preprocessing is a separate algorithm with its own recurrence ($T_{\text{sort}}(n) = 2 T_{\text{sort}}(n/2) + \Theta(n)$ for merge-sort-like sorting).

Task 7 - Solving the Recurrence

7.1 Method 1: Iteration (repeated substitution) with $n = 2^k$

This method is applicable because the recurrence reduces the argument by the same factor in every step, so the expansion has a visible pattern. For $n = 2^k$ the floor disappears:

$$\begin{aligned} T(2^k) &= T(2^{(k-1)}) + 1 \\ &= [T(2^{(k-2)}) + 1] + 1 = T(2^{(k-2)}) + 2 \\ &= [T(2^{(k-3)}) + 1] + 2 = T(2^{(k-3)}) + 3 \\ &\dots \\ &= T(2^{(k-i)}) + i \quad (\text{after } i \text{ expansions}) \\ \text{Stop when } 2^{(k-i)} &= 1, \text{ i.e. } i = k: \\ T(2^k) &= T(1) + k \quad \text{and} \quad T(1) = T(0) + 1 = 1 \\ \Rightarrow T(2^k) &= k + 1 = \log_2(n) + 1 \end{aligned}$$

7.2 Method 2: Substitution - proof for every n (not just powers of 2)

Claim: $T(n) = \lfloor \log_2 n \rfloor + 1$ for all $n \geq 1$. Proof by induction. Base: $n = 1$ gives $T(1) = T(0) + 1 = 1 = \lfloor \log_2 1 \rfloor + 1$. Step: let $n \geq 2$ and $k = \lfloor \log_2 n \rfloor$, so $2^k \leq n < 2^{(k+1)}$ and $k \geq 1$. Then $2^{(k-1)} \leq \lfloor n/2 \rfloor < 2^k$, hence $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = k - 1$. By the induction hypothesis $T(\lfloor n/2 \rfloor) = (k-1) + 1 = k$, and therefore $T(n) = T(\lfloor n/2 \rfloor) + 1 = k + 1 = \lfloor \log_2 n \rfloor + 1$. QED.

7.3 Method 3: Master Theorem (cross-check)

The Master Theorem applies to recurrences of the form $T(n) = a T(n/b) + f(n)$ with $a \geq 1$, $b > 1$ and f asymptotically positive; the floor in $\lfloor n/2 \rfloor$ does not change the result. Here $a = 1$, $b = 2$, $f(n) = \Theta(1)$. Compute $n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^{(\log_b a)})$, Case 2 of the Master Theorem applies and

$$T(n) = \Theta(n^{(\log_b a)} * \log n) = \Theta(\log n)$$

7.4 Interpretation

- The three methods agree: the worst-case number of comparisons is $\lfloor \log_2 n \rfloor + 1$ and the running time is $\Theta(\log n)$.
- Doubling the database adds exactly one more comparison. Growing from 1,000 to 1,000,000 records (x1000) raises the bound from 10 to 20 comparisons (measured worst-case counts in Task 9: 10 and 20).
- The recursion depth equals the number of comparisons, so at most 20 stack frames are needed even for 1,000,000 records - recursion is safe and the $O(\log n)$ stack space is negligible.
- The recurrence describes the search only. It says nothing about the $\Theta(n \log n)$ sorting cost, which therefore has to be added separately (Task 3).

Task 8 - Implementation and Correctness Testing

8.1 Implementation and test set-up

- **Language / environment:** Python 3.12.3, Linux x86_64; only the standard library is needed to run the algorithms (pandas/matplotlib only for graphs).
- **Implemented algorithms:** `src/algo1_linear_search.py` (iterative loop), `src/algo2_recursive_binary_search.py` (recursive function `binary_search` exactly as in Task 1,

preceded by sorted()), src/algo3_hash_table.py (own hash table with separate chaining, $h(\text{key}) = \text{key} \bmod m$, m prime $\geq n/0.75$). The pseudocode of Task 1 corresponds line by line to these functions, and the counters record the basic operations identified in Task 2.

- **Datasets:** src/dataset_generator.py with seed 1311 creates data/students_1000.csv, _10000, _100000 and _1000000 (header: registration_number). Each file holds n distinct, unsorted 9-digit registration numbers drawn with random.Random(1311).sample from [191,000,000; 193,000,000); the key 192511311 is inserted exactly once at a pseudo-random position and 192511312 is guaranteed absent. Regenerating the files gives byte-identical output (verified with md5sum).
- **Same data for all algorithms:** all three programs load the same CSV file for each size (loading time is excluded from the measurements); binary search sorts a copy of the data and the hash table is built from the same list.
- **Consistent test conditions:** same machine and interpreter run for all algorithms (benchmark.py), garbage collection disabled during timing, time measured with time.perf_counter(), each timing is the average over repeated calls and the fastest of 3 trials is reported (details in README).
- **Search keys:** FOUND key 192511311 and NOT-FOUND key 192511312 for every size.

8.2 Test cases

Table 8.1 - Test cases (python src/benchmark.py --correctness)

Test	Purpose	Expected
Key 192511311 in all four datasets	Successful search	FOUND by all three; $A[\text{index}] = \text{key}$
Key 192511312 in all four datasets	Unsuccessful search	NOT FOUND by all three
First element (linear best case)	Best case	1 comparison
Last element (linear worst case)	Worst case	n comparisons
Middle element of sorted array (binary best case)	Best case	1 comparison
Key absent - comparison bounds	Worst case counts	linear = n ; binary $\leq \lfloor \log_2 n \rfloor + 1$
100 random present + 100 random absent keys per dataset	General correctness	All three agree with Python set membership
Empty list, single element (present/absent), small unsorted list	Edge cases	Correct result, no exceptions

8.3 Actual test output

```
=====
DAA Assignment 1 - Task 8 - Correctness Tests
Name       : Natesa Krishnan V
Registration No.: 192511311
Group      : A (Dataset seed: 1311)
=====

... (datasets n = 1,000 / 10,000 / 100,000 omitted here; full output in results/logs and screenshots/) ...

--- Dataset n = 1,000,000 ---
[PASS] 192511311 FOUND by all three (linear idx=256093, binary idx=755937, hash idx=256093)
[PASS] 192511312 NOT FOUND by all three
[PASS] Linear not-found comparisons == n (1000000 == 1000000)
[PASS] Binary not-found comparisons <= floor(log2 n)+1 (20 <= 20)
[PASS] Linear best case: first element found with 1 comparison (got 1)
[PASS] Linear worst case: last element found with n comparisons (got 1000000)
[PASS] Binary best case: middle element found with 1 comparison (got 1)
[PASS] 200 random keys (100 present + 100 absent): all three agree with set membership

--- Edge cases ---
[PASS] Empty list: linear/binary/hash all NOT FOUND
[PASS] Single element, present
[PASS] Single element, absent
[PASS] Small unsorted list: every element found, neighbours absent
=====

TOTAL: 36 passed, 0 failed
ALL TESTS PASSED
```

Result: TOTAL: 36 passed, 0 failed. All three algorithms return identical results on every dataset, FOUND for 192511311 and NOT FOUND for 192511312. The measured best/worst-case counts agree with the formulas of Task 4. Screenshot files: screenshots/task08_correctness_test.png.

```

=====
DAA Assignment 1 - Benchmark (all algorithms x all dataset sizes)
Name      : Natesa Krishnan V
Registration No.: 192511311
Group     : A (Dataset seed: 1311)
=====

#### DATASET SIZE n = 1,000 ####
Algorithm  Key      Result      Comps/Ops  Preproc(ms)  Search(ms)  Total(ms)
Linear Search 192511311 FOUND         47      0.0000      0.001991      0.0020
Linear Search 192511312 NOT FOUND      1000     0.0000      0.048001      0.0480
Recursive Binary Search 192511311 FOUND          7      0.0620      0.001251      0.0633
Recursive Binary Search 192511312 NOT FOUND      10      0.0620      0.001812      0.0638
Hash Table Search 192511311 FOUND          2      0.2115      0.000179      0.2116
Hash Table Search 192511312 NOT FOUND          2      0.2115      0.000178      0.2116
All three algorithms agree on results: True

#### DATASET SIZE n = 10,000 ####
Algorithm  Key      Result      Comps/Ops  Preproc(ms)  Search(ms)  Total(ms)
Linear Search 192511311 FOUND        8849     0.0000      0.461671      0.4617
Linear Search 192511312 NOT FOUND     10000     0.0000      0.575705      0.5757
Recursive Binary Search 192511311 FOUND         14      1.2306      0.002217      1.2328
Recursive Binary Search 192511312 NOT FOUND         14      1.2306      0.002364      1.2330
Hash Table Search 192511311 FOUND          2      2.7276      0.000174      2.7278
Hash Table Search 192511312 NOT FOUND          2      2.7276      0.000173      2.7278
All three algorithms agree on results: True

#### DATASET SIZE n = 100,000 ####
Algorithm  Key      Result      Comps/Ops  Preproc(ms)  Search(ms)  Total(ms)
Linear Search 192511311 FOUND       41117     0.0000      2.419260      2.4193
Linear Search 192511312 NOT FOUND    100000     0.0000      5.480760      5.4808
Recursive Binary Search 192511311 FOUND         17     15.6162      0.002745      15.6189
Recursive Binary Search 192511312 NOT FOUND         17     15.6162      0.003161      15.6194
Hash Table Search 192511311 FOUND          2     36.7148      0.000225      36.7150
Hash Table Search 192511312 NOT FOUND          1     36.7148      0.000276      36.7151
All three algorithms agree on results: True

#### DATASET SIZE n = 1,000,000 ####
Algorithm  Key      Result      Comps/Ops  Preproc(ms)  Search(ms)  Total(ms)
Linear Search 192511311 FOUND     256094     0.0000     16.125600      16.1256
Linear Search 192511312 NOT FOUND  1000000     0.0000     54.806460      54.8065
Recursive Binary Search 192511311 FOUND         18     260.4748      0.004598      260.4794
Recursive Binary Search 192511312 NOT FOUND         20     260.4748      0.004769      260.4796
Hash Table Search 192511311 FOUND          2    485.5601      0.000379      485.5605
Hash Table Search 192511312 NOT FOUND          2    485.5601      0.000349      485.5604
All three algorithms agree on results: True

```

Figure 8.1 - Terminal-style image of the test output (rendered from the captured log; see README for how to take a live terminal screenshot).

Task 9 - Experimental Performance

All values below are copied from results/results.csv, produced by python src/benchmark.py. Times are per single query; preprocessing is the one-off cost of sorting (binary search) or building the table (hash). 'Ops' for the hash table = 1 hash computation + key comparisons in the chain.

Table 9.1 - Number of key comparisons / operations

n	Linear comparisons (found / not found)	Binary comparisons (found / not found)	Hash operations (found / not found)	Binary bound floor(log2 n)+1	Hash load factor
1,000	47 / 1,000	7 / 10	2 / 2	10	0.735
10,000	8,849 / 10,000	14 / 14	2 / 2	14	0.750
100,000	41,117 / 100,000	17 / 17	2 / 1	17	0.750
1,000,000	256,094 / 1,000,000	18 / 20	2 / 2	20	0.750

Table 9.2 - Search time per query (preprocessing excluded)

n	Linear search time ms (found / not found)	Binary search time ms (found / not found)	Hash search time ms (found / not found)
1,000	0.0019 / 0.0535	0.001420 / 0.001307	0.000246 / 0.000231
10,000	0.4988 / 0.6104	0.001863 / 0.002519	0.000135 / 0.000135
100,000	1.9345 / 4.2221	0.002879 / 0.003194	0.000134 / 0.000082
1,000,000	8.2486 / 37.0278	0.002307 / 0.002728	0.000133 / 0.000139

Table 9.3 - Preprocessing and total time for ONE query (not-found key), ms

n	Linear preproc. (ms)	Binary: sort (ms)	Hash: build (ms)	Linear total, 1 query	Binary total, 1 query	Hash total, 1 query
1,000	0	0.0633	0.2400	0.0535	0.0646	0.2402
10,000	0	1.2865	2.8718	0.6104	1.2891	2.8719
100,000	0	18.5323	40.5688	4.2221	18.5355	40.5688
1,000,000	0	278.1463	600.8523	37.0278	278.1490	600.8524

Total time = preprocessing + search time. If only a single search is made, the preprocessing dominates and linear search is cheapest; the picture changes once the preprocessing is shared by many searches (Figure 9.3).

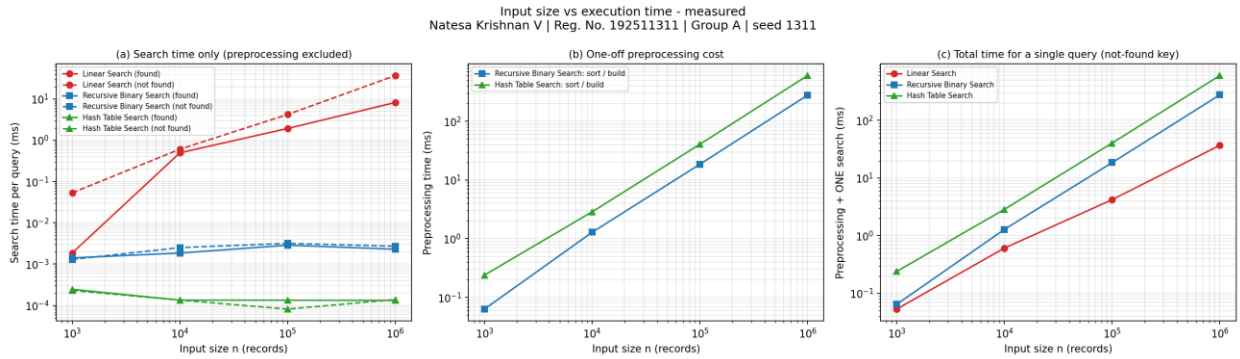


Figure 9.1 - Input size vs execution time (log-log axes), generated from results.csv

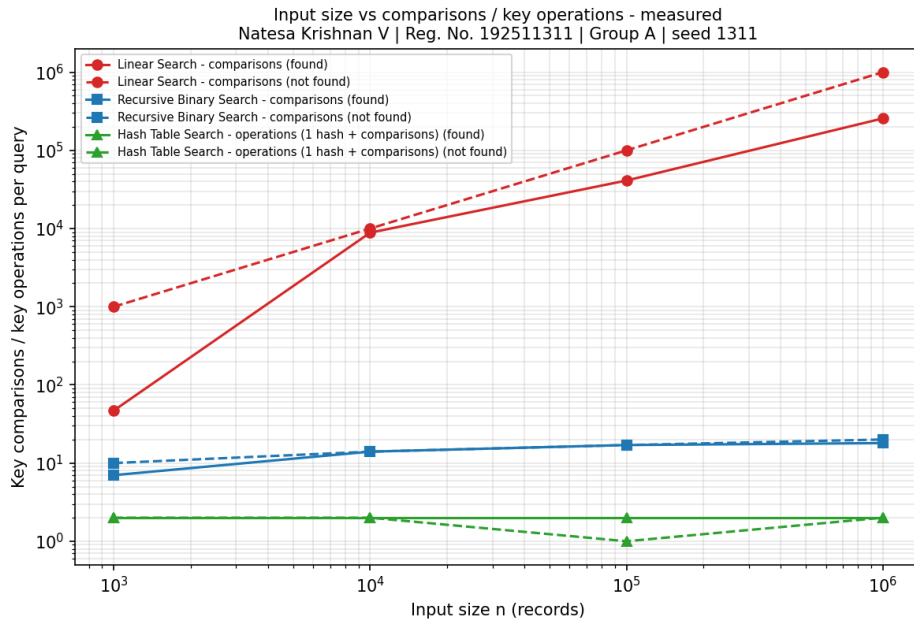


Figure 9.2 - Input size vs key comparisons / operations (log-log axes)

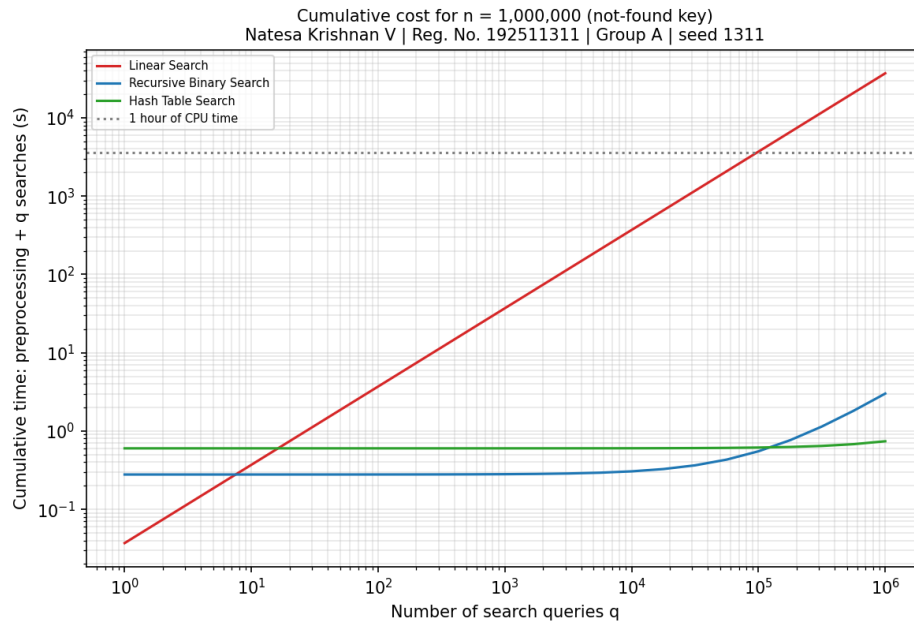


Figure 9.3 - Cumulative cost of preprocessing + q searches for $n = 1,000,000$

Observations. (1) Linear-search comparison counts equal 1,000 / 10,000 / 100,000 / 1,000,000 for the absent key and (position + 1) for the present key. (2) Binary-search counts are 10 / 14 / 17 / 20 in the worst case, exactly $\text{floor}(\log_2 n) + 1$. (3) The hash table needs 1-2 operations regardless of n . (4) Measured search times mirror these counts: the linear search grows by about three orders of magnitude, the other two stay in the micro/sub-microsecond range. (5) The preprocessing of both binary search and hash search grows with n and is far larger than one search. Screenshots of the benchmark runs: screenshots/task09_runtime_1000.png, _10000.png, _100000.png, _1000000.png.

Task 10 - Theory vs Experimental Behaviour

Table 10.1 - Predicted and measured growth factors

Quantity	Theory: growth for n 1,000 $\rightarrow$ 1,000,000	Measured growth	Agreement
Linear: comparisons (not found)	$\times 1000$ (n)	$\times 1,000$	exact match
Linear: search time (not found)	$\times 1000$ (n)	$\times 692$ ($t \sim n^{0.95}$)	close to linear; slightly sub-linear
Binary: comparisons (not found)	$\times 2.0$ ($\log n$)	$\times 2.0$	exact match
Binary: search time (not found)	$\times 2.0$ ($\log n$)	$\times 2.1$	same order; noisy at 1-3 microseconds
Binary: sort time	$\times 2,000$ ($n \log n$)	$\times 4,392$	larger than predicted
Hash: operations (not found)	$\times 1$ (constant)	$\times 1.0$	constant (1 or 2)
Hash: search time (not found)	$\times 1$ (constant)	$\times 0.60$	constant order (~ 0.1 microsecond)
Hash: build time	$\times 1000$ (n)	$\times 2,504$	larger than predicted

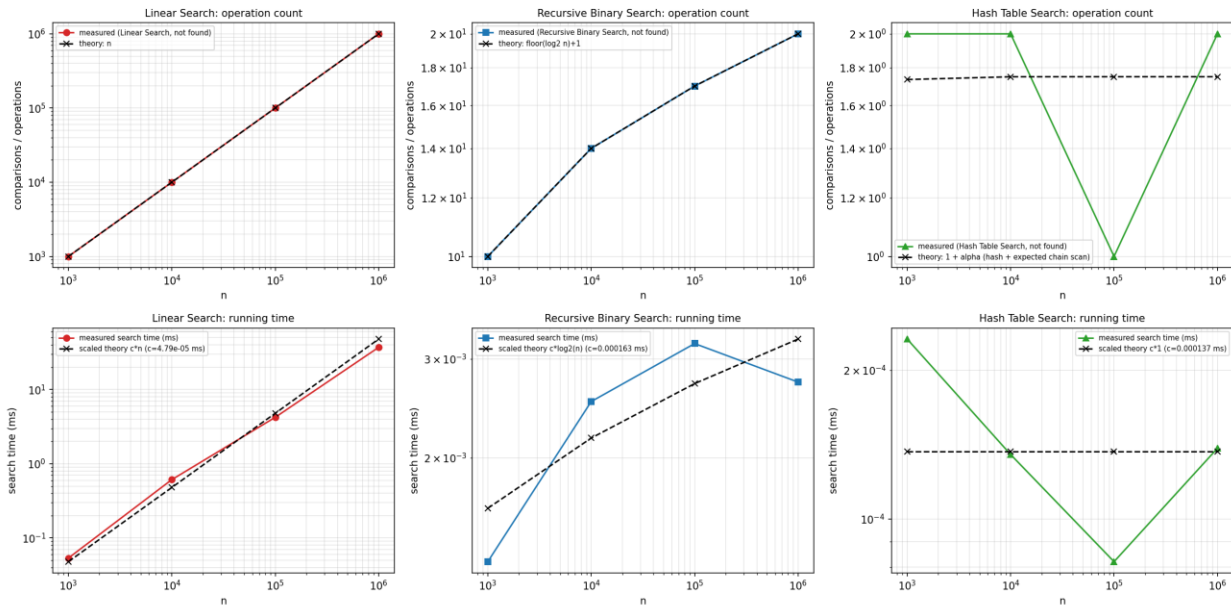


Figure 10.1 - Measured comparison counts and running times against the theoretical curves (scaled by a fitted constant)

10.1 Does the observed growth agree with the theory?

- Operation counts agree exactly. Linear search performs n comparisons for the absent key and $\text{position} + 1$ for the present one; binary search needs exactly $\text{floor}(\log_2 n) + 1$ comparisons in its worst case (10, 14, 17, 20), the successful search needs between 7 and 18; hash search needs 1-2 operations. These counts are machine-independent, so they validate Tasks 3-7 directly.
- Running times agree in shape. The linear search time rises by a factor of about 692 for a 1000-fold larger n (empirical exponent 0.95 against the theoretical 1). Binary and hash search times stay within a few microseconds and a fraction of a microsecond respectively, consistent with $\Theta(\log n)$ and $\Theta(1)$.
- Scaling: the hash table scales best for searching (flat), binary search next (adds one comparison per doubling), linear search worst. For the total cost including preprocessing, the ranking depends on q (Task 11).

10.2 Differences between prediction and observation and their causes

- Preprocessing grows faster than n or $n \log n$. The sort took x4,392 longer (predicted x2,000) and the hash build x2,504 longer (predicted x1000) for 1000x more records. Reasons: the 1,000-record data fits in the CPU cache whereas 1,000,000 Python int objects, tuples and lists do not (cache misses and memory allocation per element), and the big-O model ignores memory-hierarchy effects.
- Hash build is more expensive than the sort at every size. Both are linear-ish in theory, but building the table creates a tuple and a list per record and allocates $m = 1,333,357$ buckets in pure Python, whereas `sorted()` runs in optimised C. This is a property of the implementation language, not of the algorithm.
- Measured search times of binary and hash search are noisy. Their true cost is 0.1-3 microseconds, so the timer call, function-call overhead of the Python interpreter and OS scheduling are comparable to the measurement itself; this is why the measured binary time at 1,000,000 records can be below the time at 100,000 although the comparison count is higher. The comparison counts are the reliable evidence; the times show the order of magnitude.
- Found-key times depend on where the key happens to be. The key is located at index 46 ($n=1,000$), 8,848 ($n=10,000$), 41,116 ($n=100,000$), 256,093 ($n=1,000,000$), so found-key times follow the position (at $n = 1,000,000$ the key is at about 25.6 % of the array), and the per-comparison cost varies with caching and branch prediction (about 37 ns per comparison for the absent key at $n = 1,000,000$).

- **Hash operations vary between 1 and 2.** An empty bucket gives 0 comparisons (1 operation); a bucket holding the key (or another key) gives 1 comparison (2 operations). The theoretical expectation is $\alpha \sim 0.75$ comparisons for an absent key; a single measured key can only be 0 or 1, so one run cannot reproduce the average, which is a property of the average-case definition.
- **Why the chains are so short at $n = 1,000,000$.** The generated keys lie in a range of 2,000,000 values and the table size is $m = 1,333,357$ (about 1.33 million), so $\text{key mod } m$ can map at most two different keys to the same bucket; the longest chain is therefore bounded by 2 for this dataset by construction. The measured hash-table behaviour is thus a favourable but legitimate case for these registration numbers; keys with a different structure (for example many keys that are multiples of m) would produce longer chains, up to the $\Theta(n)$ worst case of Task 4.
- **Hardware and interpreter effects.** Absolute times depend on the machine, Python version and background load; only growth trends are transferable. Garbage collection was disabled during timing and the fastest of three trials is reported to reduce noise.

Task 11 - Final Design Recommendation

Table 11.1 - Total time for q searches on $n = 1,000,000$ records (measured preprocessing + measured mean search time of the found and not-found keys)

Number of searches q	Linear Search (s)	Binary Search: sort + q (s)	Hash Table: build + q (s)
1	0.0226	0.2781	0.6009
10	0.2264	0.2782	0.6009
100	2.2638	0.2784	0.6009
1,000	22.6382	0.2807	0.6010
10,000	226.3822	0.3033	0.6022
100,000	2,263.8215	0.5299	0.6145
1,000,000	22,638.2150	2.7956	0.7369

Break-even points computed from the measurements ($n = 1,000,000$): binary search overtakes linear search after about 12 searches (sort time 278.1 ms divided by the saving of 22.636 ms per search); the hash table overtakes linear search after about 27 searches (build time 600.9 ms / 22.638 ms); the hash table overtakes binary search after about 135,505 searches, because it has a higher build cost (600.9 ms vs 278.1 ms) but a lower per-search cost (0.136 microseconds vs 2.517 microseconds).

Recommendation: Hash Table Search

For a university record system that must answer registration-number lookups repeatedly on a database of up to 1,000,000+ records, I recommend the hash table, with the sorted array and recursive binary search as the memory-lean alternative and linear search only for very small or one-off look-ups.

- Why it is more appropriate than the alternatives. The record system is a lookup service: the data are searched many times and change rarely. Linear search pays $\Theta(n)$ on every request (mean 22.64 ms per request at $n = 1,000,000$ in our measurements); binary search pays $\Theta(n \log n)$ once and then $\Theta(\log n)$ per request; the hash table pays $\Theta(n)$ once and then $\Theta(1)$ expected per request.
- Theoretical efficiency. Expected $O(1)$ per search with $\alpha \sim 0.75$; build $\Theta(n)$; $O(1)$ per insertion of a new record, whereas a sorted array needs an $O(n)$ insertion or a re-sort.
- Experimental evidence. At $n = 1,000,000$ the hash table needed 1-2 operations and about 0.14 microseconds per search against 2.52 microseconds for binary search (18-20 comparisons) and 22.64 ms for linear search (256,094 and 1,000,000 comparisons).
- **Preprocessing cost.** Building the table took 601 ms for 1,000,000 records (2.2 times the sorting time of 278 ms). This is repaid after about 27 searches compared with linear search, i.e. immediately for any real workload.
- **Trade-offs and limitations.** (a) Extra memory $\Theta(n)$ for the table and chains; (b) the $O(1)$ bound is an expected bound that relies on a good hash function and a bounded load factor (the worst case is $\Theta(n)$; the measured longest chain was only 2); (c) no ordered traversal or range queries - binary

search on the sorted array supports these; (d) for a single query or very small n (our $n = 1,000$ data: linear total 0.0535 ms vs hash total 0.2402 ms), the preprocessing is not worth it and linear search is better; (e) for 1,000 searches or more at $n = 1,000,000$, both binary search and hash search are orders of magnitude better than linear search, but they differ from each other by only a fraction of a second in total - the hash table's decisive advantages are $O(1)$ updates and a flat per-query cost, not a large absolute saving over binary search.

- **Does it stay practical as the database grows?** Yes: the search cost does not grow with n , the build and memory grow linearly, and the table can be rehashed (to keep α bounded) at amortised $O(1)$ per insertion. Linear search does not remain practical (its cost grows 1000-fold from 1,000 to 1,000,000 records), and binary search remains practical but needs an $O(n)$ merge or an $O(n \log n)$ re-sort whenever records are added.

Task 12 - Day-4 Design Challenge

Changed requirement: 50,000 search requests per hour on a database of 1,000,000 records; new records are added only once per day.

A. Impact Analysis

- **What has changed.** The original brief gave no workload; it now specifies a sustained load of 50,000 searches per hour (13.9 requests per second, one request every 72 ms on average, 1,200,000 searches per day) on a fixed $n = 1,000,000$, with updates only once a day.
- **Assumptions affected.** The earlier analysis implicitly compared the cost of ONE search (or a few). That assumption is now false: the number of searches q is huge compared with n , so the preprocessing cost is paid once and shared by about 1,200,000 searches per day, and it is the per-search cost that decides the design. Also, the data are effectively static between daily updates, which removes the main argument against structures that are expensive to update.
- **Why search frequency matters.** Total daily cost = preprocessing + $q \times$ (cost per search). For linear search the second term is $q \times \Theta(n)$; for the other two it is $q \times \Theta(\log n)$ or $q \times \Theta(1)$. With $q = 1,200,000$ per day the second term dominates everything, which makes per-search efficiency the critical measure.
- **Why the low update frequency matters.** An index that is expensive to build or modify (a sorted array or a hash table) needs to be rebuilt or patched only once a day, so its $\Theta(n)$ or $\Theta(n \log n)$ construction is amortised over about 1.2 million searches: 0.60 s (hash) or 0.28 s (sort) per day is negligible against 86,400 s.

B. Evaluation of the Original Design

Table 12.1 - Expected load of the three designs under the new requirement (computed from measured times)

Algorithm ($n = 1,000,000$)	Mean search time (ms)	Worst measured (not-found) ms	CPU seconds per hour (50,000 searches)	Utilisation of one core	CPU seconds per day incl. daily rebuild	Preprocessing
Linear Search	22.638215	37.027822	1,131.911	31.4420 %	27,165.86	none
Recursive Binary Search	0.002517	0.002728	0.126	0.0035 %	3.30	278.1 ms once per day
Hash Table Search	0.000136	0.000139	0.007	0.0002 %	0.76	600.9 ms once per day

- Linear search. Needs 1,132 CPU-seconds per hour on average (31.4 % of one core) and 1,851 s (51.4 %) if most requests are for absent records; each request takes up to 37.0 ms while requests arrive every 72 ms on average, so bursts of requests would queue. Its cost also grows linearly with database growth. It is technically feasible on a single core in this Python measurement, but with little headroom and no scalability.
- Binary search (sorted array). Preprocessing of 278 ms once per day is acceptable; searches then use about 2.5 microseconds, a negligible 0.00350 % of one core. A strong candidate.

- Hash table (original recommendation). Build time of 601 ms once per day is acceptable (it is 0.0007 % of a day); searches use about 0.14 microseconds, with the lowest, flat per-query cost. New records can be inserted in $O(1)$ each.
- Among the three studied approaches the hash table has the lowest cost for $q = 1.2$ million searches per day (daily total 0.76 s vs 3.30 s for binary search and 27,166 s for linear search). The difference between hash and binary search is small in absolute terms; what separates them is the cost of daily maintenance and flat latency under growth.

C. Adaptation or Redesign - Decision: MODIFY

Decision: MODIFY the original solution. The core algorithm - hash table search - is RETAINED because the changed requirement strengthens the argument for it; what changes is HOW the structure is built, kept and updated (the operating strategy), not the search algorithm. A full REDESIGN to a different data structure is not justified by the evidence.

Revised strategy:

- Build the hash table once at system start-up (or in an off-peak window) and keep it resident in memory; never rebuild it per request. Preprocessing is then a one-off cost, measured at 601 ms for $n = 1,000,000$.
- Handle the daily update incrementally: insert the k new records in $O(k)$ time (each insertion is an $O(1)$ chain append) instead of rebuilding in $\Theta(n)$. Keep the load factor bounded (e.g. $\alpha \leq 0.75$) by rehashing into a larger prime-sized table when the database grows past the threshold; rehashing costs $\Theta(n)$ but happens rarely, so insertion remains amortised $O(1)$.
- Because the data change only once a day, the table is read-only between updates: concurrent searches need no locking. The daily update can be done on a background copy and swapped in atomically, so no request waits for it.
- Keep the records in sorted order on disk (or a sorted copy) as a fall-back and for range or ordered queries, which a hash table cannot answer; this copy is re-sorted or merged once a day ($O(n \log n)$ or $O(n + k \log k)$).

Table 12.2 - What changes in the modified design

Aspect	Original design	Modified design
Search algorithm	Hash table search	Hash table search (retained)
When the table is built	Whenever searching starts (cost charged to the search task)	Once at start-up, then kept in memory
Daily new records	Rebuild: $\Theta(n)$	Incremental insert: $\Theta(k)$ for k new records; periodic rehash if α too large
Cost per search	$\Theta(1)$ expected	$\Theta(1)$ expected (unchanged)
Cost per day ($n = 1,000,000$, 1.2 M searches)	build + $q \times \Theta(1)$	insert $k + q \times \Theta(1) =$ about 0.16 s of search work (measured) instead of 27,166 s for linear search
Concurrency	Not addressed	Read-only between updates; update on a copy and swap

Effect on computational complexity: the total cost of the first day changes from $\Theta(n) + q \times \Theta(1)$ to the same, and the cost of each subsequent day falls from $\Theta(n + q)$ (rebuild every day) to $\Theta(k + q)$ (incremental insertion), where $k \ll n$ is the number of records added that day. Searching stays $\Theta(1)$ expected; memory stays $\Theta(n)$.

D. Evidence-Based Defence

- Mathematical complexity. Daily cost with $q = 1,200,000$: linear search $\Theta(qn) =$ about 1.2×10^{12} key comparisons for $n = 10^6$ (if every request has to scan the whole array; about 3.1×10^{11} for found keys at the measured average position of 25.6 %); binary search $\Theta(n \log n + q \log n) =$ about 2×10^7 (sort) + 2.4×10^7 (searches) comparisons; hash table $\Theta(n + q) =$ about 10^6 insertions + about 2.4×10^6 operations. The hash table has the lowest order of growth.

- Experimental results. Measured at $n = 1,000,000$: mean search time 22.64 ms (linear), 2.52 microseconds (binary), 0.136 microseconds (hash); preprocessing 278 ms (sort) and 601 ms (build). These give the daily totals of Table 12.1 and the break-even points of Task 11 (27 searches against linear search; 135,505 against binary search), both of which are passed within the first hours of a day at 50,000 searches per hour (the second after about 163 minutes).
- Search frequency and dataset size. With 13.9 requests per second and $n = 10^6$, a linear scan occupies 31 % of a core on average; any growth in the database or request rate (or slower hardware or a disk-based store) would saturate it. Hash and binary search are practically insensitive to this load.
- Preprocessing cost and update frequency. One build per day (0.60 s) is negligible, and incremental inserts make even this unnecessary; a once-a-day update is precisely the situation in which an index pays off most.
- Scalability. If the database doubles, the search cost of the hash table stays constant, binary search adds one comparison, and linear search doubles.
- Trade-offs accepted. $\Theta(n)$ extra memory (the table, chains and tuples) and no ordered/range queries (a sorted fall-back copy covers these); expected rather than guaranteed $O(1)$ (mitigated by prime table size and bounded load factor; worst chain measured: 2); and the measured advantage over binary search is small in absolute terms, so binary search on a sorted array is a legitimate lower-memory alternative if updates are only ever batched and range queries are needed.

Conclusion: MODIFY - retain the hash-table search, build it once and maintain it incrementally, because its per-search cost is constant, its preprocessing cost is amortised over about 1.2 million searches per day, daily insertions are cheap, and the evidence shows it avoids the unacceptable per-request cost of linear search.

Appendix - Reproducing the Results

```
python src/dataset_generator.py      # creates data/students_*.csv (seed 1311)
python src/algo1_linear_search.py    # results/linear_search_results.csv
python src/algo2_recursive_binary_search.py # results/binary_search_results.csv
python src/algo3_hash_table.py       # results/hash_table_results.csv
python src/benchmark.py --correctness # Task 8 tests
python src/benchmark.py              # results/results.csv (combined)
python src/generate_graphs.py        # results/graphs/*.png
```

Limitations of the measurements: single machine, Python implementation (absolute times would differ in C/C++ or Java and on other hardware), timings of 0.1-3 microseconds are close to the resolution of the method, and the datasets are random so the average-case assumptions hold; structured registration numbers could produce more hash collisions (for this dataset the modulus bounds the chain length at 2 for $n = 1,000,000$). The tables in this report belong to one benchmark run; re-running the benchmark produces slightly different times (the comparison counts are identical).